Models and Tools for Cyber-Physical Systems

Digital Written Exam, August the 21st 2023, 10:00-14:00

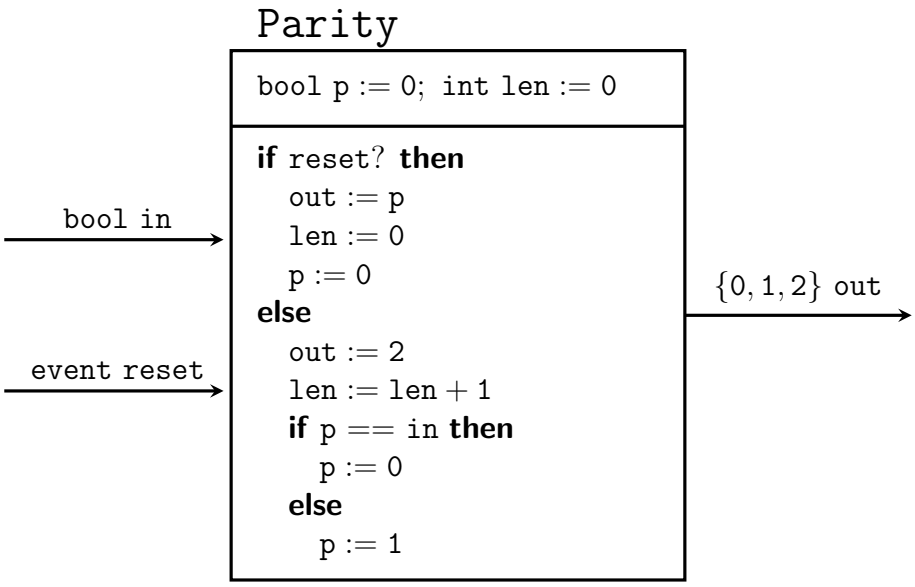
Please read the following before solving the exercises.

- This exam contains 6 exercises, worth 75 points in total. Each exercise is compulsory. The solution has to be composed in English. If you believe that the assignment wording is ambiguous or erroneous, then write down what additional assumption you are using and outline your reasons.
- In some exercises you are asked to extend incomplete UPPAAL and Python files. Solutions have to be uploaded to Digitaleksamen in the form of UPPAAL, Python, pdf or text files. For each of your solutions, ensure that it is clear which exercise is solved.
- The exam is open book but asking/receiving help from others is not allowed.
- In case of emergencies: Students can contact the instructors during the exam by approaching the study secretary.

Last but not least, good luck!

Exercise 1: Synchronous Components 12 Points

Consider the synchronous component `Parity` given as a block diagram below.



(a) Below we wrote the initial state and a sequence of inputs. Fill in the blanks to obtain a proper prefix of a corresponding execution.

$$(0,0) \xrightarrow{(1,\bot)/} \underline{\qquad\qquad} \xrightarrow{(1,\bot)/} \underline{\qquad\qquad}$$

$$\xrightarrow{(0,\bot)/} \underline{\qquad\qquad} \xrightarrow{(1,\top)/} \underline{\qquad\qquad}$$

Hint: Recall that $\bot$ denotes that an event is not present, and $\top$ denotes that it is present. You need to provide both the missing states between the arrows and the missing outputs after the "/" on the arrows.

(b) Give a prefix of an <u>execution</u> leading to the state $(1,3)$.
(Provide the inputs, intermediate states, and outputs, as in task (a).)

(c) Describe the component `Parity` as an extended state machine.
Use the mode of the state machine to capture the value of the state variable `p`, while the state variable `len` should be updated using assignments in the mode switches.

Hint: Recall that `reset?` is used to check that the `reset` event is present, and `¬reset?` to check that it is absent.

..... Solution sketch .....

(a) $(0,0) \xrightarrow{(1,\bot)/2} (1,1) \xrightarrow{(1,\bot)/2} (0,2) \xrightarrow{(0,\bot)/2} (0,3) \xrightarrow{(1,\top)/0} (0,0)$

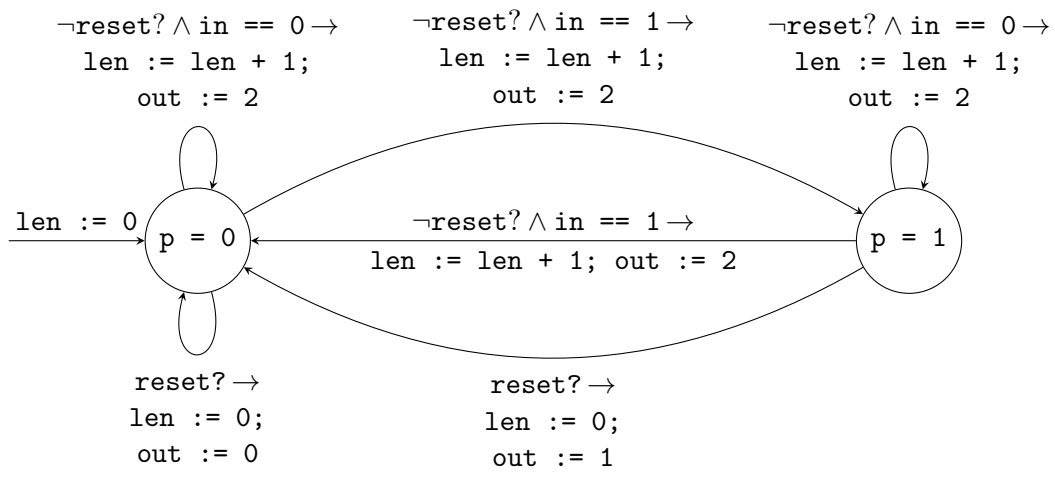
(b) The shortest solutions have length three. Below are all valid length-three input sequences.

$(1,\bot),(1,\bot),(1,\bot) \qquad (1,\bot),(0,\bot),(0,\bot) \qquad (0,\bot),(1,\bot),(0,\bot) \qquad (0,\bot),(0,\bot),(1,\bot)$

Execution corresponding to the first sequence:

$$(0,0) \xrightarrow{(1,\bot)/2} (1,1) \xrightarrow{(1,\bot)/2} (0,2) \xrightarrow{(1,\bot)/2} (1,3)$$

(c)


¬reset? ∧ in == 0 →
len := len + 1;
out := 2
¬reset? ∧ in == 1 →
len := len + 1;
out := 2
¬reset? ∧ in == 0 →
len := len + 1;
out := 2
len := 0
p = 0
¬reset? ∧ in == 1 →
len := len + 1; out := 2
p = 1
reset? →
len := 0;
out := 0
reset? →
len := 0;
out := 1

Exercise 2: Safety Monitor 8 Points

Consider a component with an output variable of type $\{0, 1, 2\}$, and the following safety property:

> The output 1 is never immediately followed by the output 0.

For example, the output sequence 0 2 1 2 0 2 1 2 ... is allowed, but the output sequence 0 1 2 <u>1 0</u> 1 2 ... is not allowed.

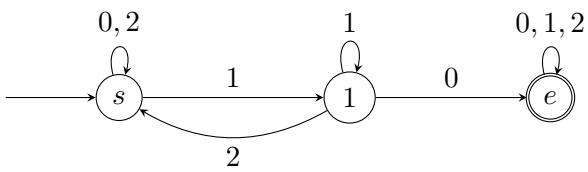
(a) Design a safety monitor, given as extended state machine, to capture the above safety property.

(b) Is the above safety property satisfied by the synchronous component `Parity` from Exercise 1?

If so, give a short argument. If not, give the prefix of a counterexample execution.

.....Solution sketch .....

(a) The intuition of the state machine below is that in state "1" the currently read string ends in 1. Thus reading a 0 there leads to the error state.

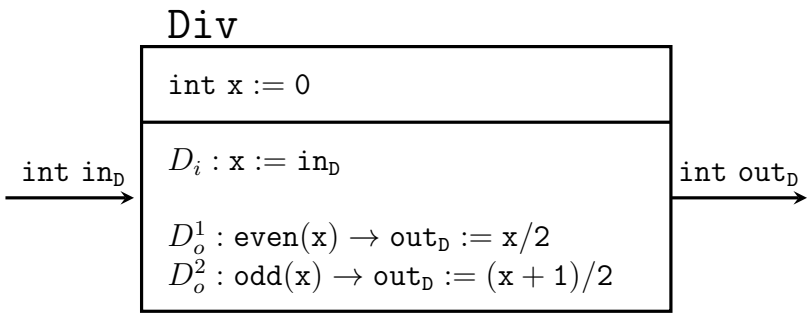


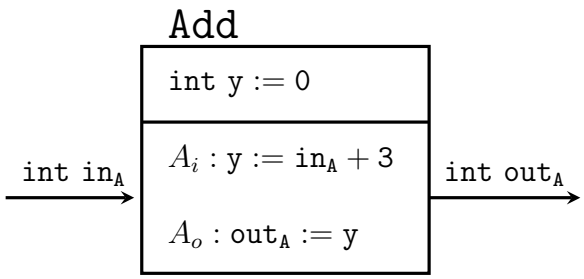
(b) The property is violated. From the state machine in the previous exercise, we see that the output subsequence 0 1 is possible by taking the two bottom transitions. The inputs "`in`" in these two bottom transitions do not matter. But first we need to get to the mode "$p = 1$". Thus there are four shortest execution prefixes of length three, where "?" can be either 0 or 1:

$$(0,0) \xrightarrow{(1,\bot)/2} (1,1) \xrightarrow{(?,\top)/1} (0,0) \xrightarrow{(?,\top)/0} (0,0)$$

Exercise 3: Asynchronous Components 10 Points

Consider the following asynchronous components `Div` and `Add` given as block diagrams.





Consider the parallel composition given by the following description:

$$\texttt{Div}[\texttt{out}_\texttt{D} \mapsto \texttt{temp}] \quad | \quad \texttt{Add}[\texttt{in}_\texttt{A} \mapsto \texttt{temp}] \quad \backslash \quad \texttt{temp}$$

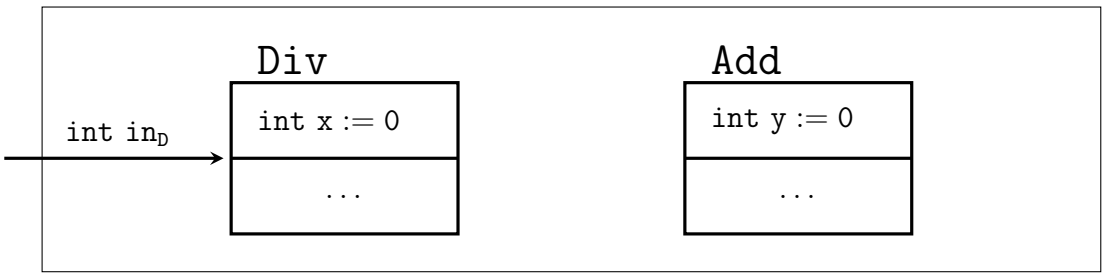
(a) Below is a partial block diagram for the parallel composition. Connect the inputs and outputs of the components correctly with edges/arrows. Inputs and outputs of the composition should leave the outer block; we have given an example for the input $\texttt{in}_\texttt{D}$.

(b) Annotate the edges you drew with the channel types and names.

(c) Give all input tasks, internal tasks, and output tasks of the parallel composition.

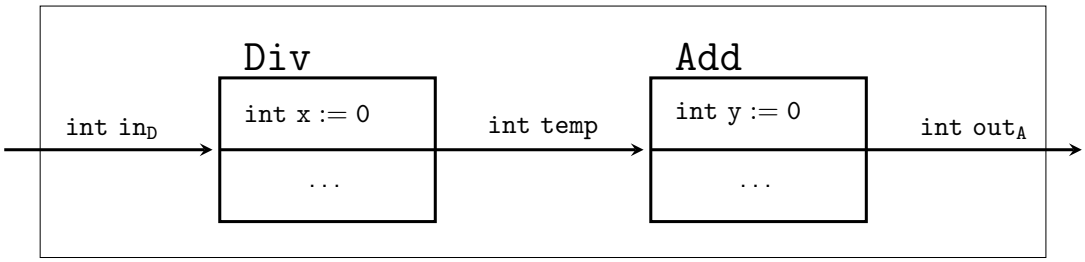
(d) Give the execution for the input sequence 2 4 5.
(Provide the intermediate states and outputs as in Exercise 1.)

Hint: The local variable `temp` is not part of the state.



. . . . . Solution sketch . . . . . .

(a) and (b)



(c)

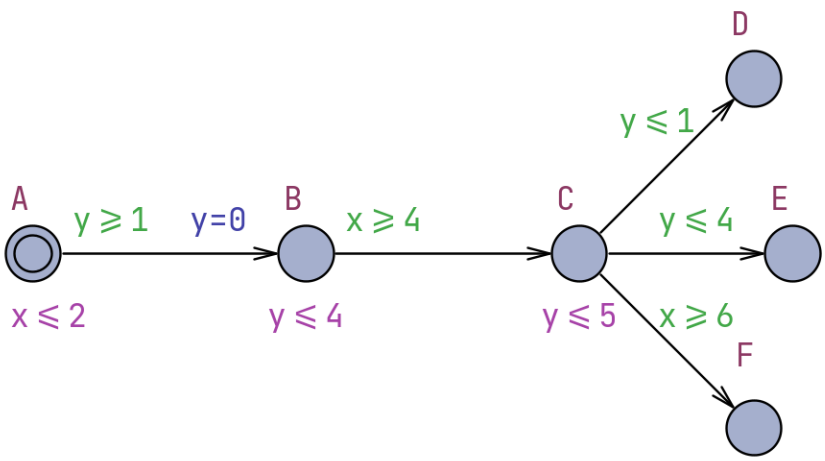
$$\begin{aligned}
\text{input: } & \mathtt{x} := \mathtt{in_D} \quad (= D_i) \\
\text{internal: } & \mathtt{even}(\mathtt{x}) \to \mathtt{temp} := \mathtt{x}/2; \mathtt{y} := \mathtt{temp} + 3 \\
& \mathtt{odd}(\mathtt{x}) \to \mathtt{temp} := (\mathtt{x}+1)/2; \mathtt{y} := \mathtt{temp} + 3 \\
\text{output: } & \mathtt{out_A} := \mathtt{y} \quad (= A_o)
\end{aligned}$$

(d) We use the state order $(\mathtt{x}, \mathtt{y})$. To read an input, we need to apply an input task. (Here this task is unique.) Thus the shortest execution (prefix) just applies the input task three times.

$$(0,0) \xrightarrow{\mathtt{in_D}\,?\,2} (2,0) \xrightarrow{\mathtt{in_D}\,?\,4} (4,0) \xrightarrow{\mathtt{in_D}\,?\,5} (5,0)$$

Exercise 4: Exploration of Timed Automata 10 Points

Consider the timed automaton $\mathcal{A}$ below with two clocks x and y.



(a) Which of the three locations D, E and F are reachable from the initial state (A, $\mathtt{x} = 0, \mathtt{y} = 0$)?

(b) For each of these three goal locations that is reachable, provide a timed transition sequence that leads to the location from the initial state.

(c) For each of the three goal locations that is reachable, what is the fastest time of reaching that location. Provide a witness timed transition sequence.

(d) Describe using difference constraints the reachable zones upon entry and after delay for the locations A, B and C.

(e) For each of the three goal locations that is NOT reachable, investigate whether it is possible to weaken the guard leading to the location, so that the location becomes reachable.
NOTE: `x<=7` is weaker than `x<=5`. Similarly, `x>=2` is weaker than `x>=4`.

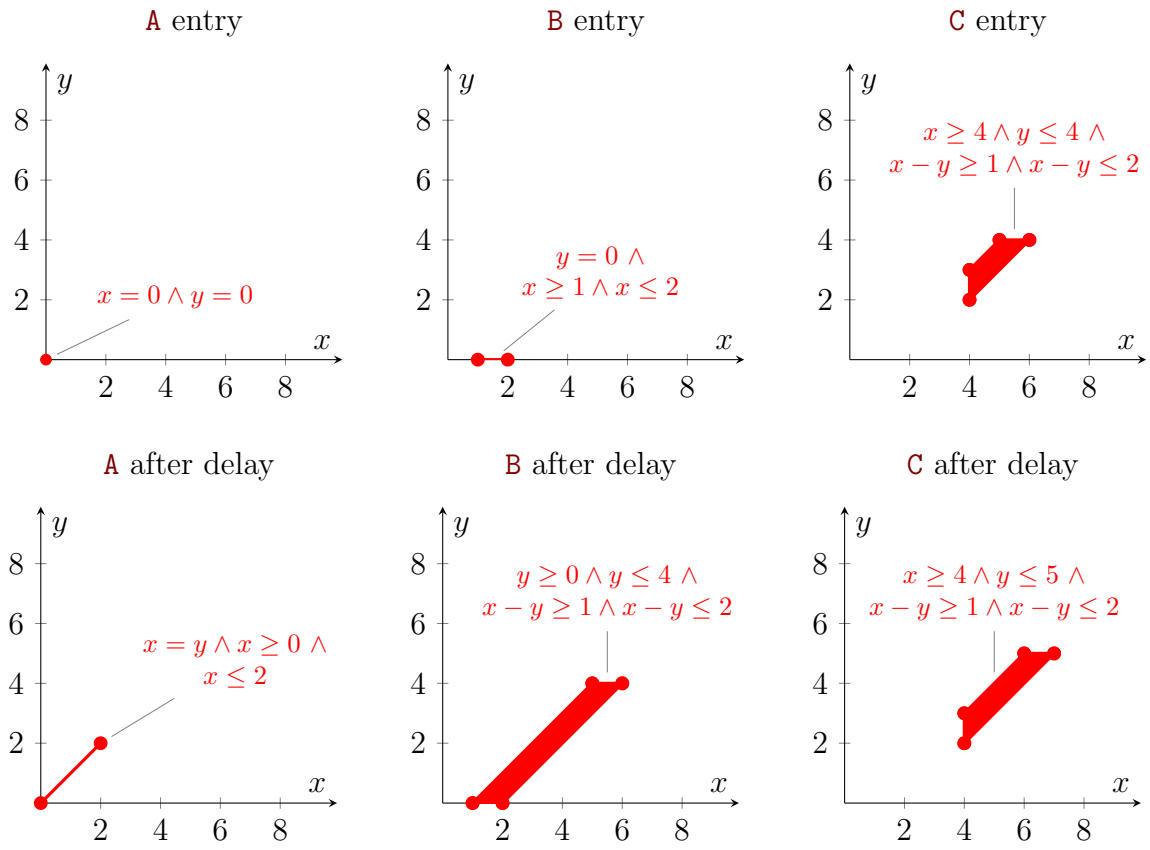
Solution sketch

(a) Both E and F are reachable.

(b) $(\mathtt{A}, x = 0, y = 0) \xrightarrow{1} (\mathtt{A}, x = 1, y = 1) \rightarrow (\mathtt{B}, x = 1, y = 0) \xrightarrow{3} (\mathtt{B}, x = 4, y = 3) \rightarrow (\mathtt{C}, x = 4, y = 3) \rightarrow (\mathtt{E}, x = 4, y = 3)$
and
$(\mathtt{A}, x = 0, y = 0) \xrightarrow{1} (\mathtt{A}, x = 1, y = 1) \rightarrow (\mathtt{B}, x = 1, y = 0) \xrightarrow{3} (\mathtt{B}, x = 4, y = 3) \rightarrow (\mathtt{C}, x = 4, y = 3) \xrightarrow{2} (\mathtt{C}, x = 6, y = 5) \rightarrow (\mathtt{F}, x = 6, y = 5)$

(c) The fastest time to reach E is 4 time units, and the fastest time to reach F is 6 time units. The witnessing sequences are above.

(d) The reachable zones upon entry and after delay:



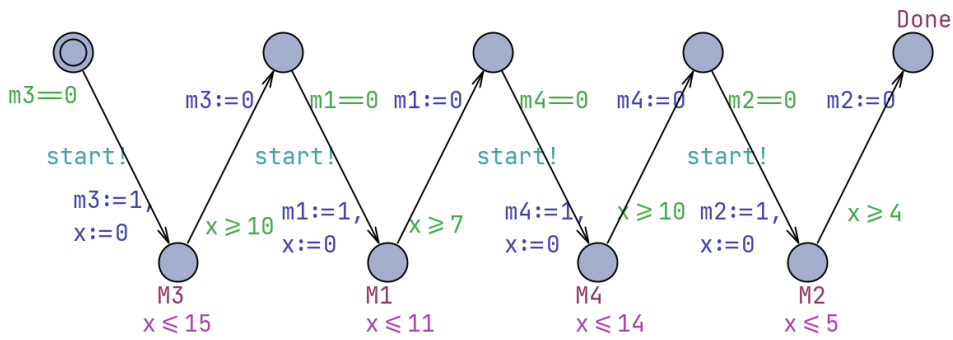
(e) Changing the guard to $y \leq 2$ will make D reachable.

Exercise 5: Flexible manufacturing system 15 Points

In this exercise we consider a flexible manufacturing system. In such a system, robots move products around two different machines. Typical machines include a drilling machine, laser cutter, painting station, and an assembly station. By changing the order of operations, different types or variations of a product can be produced.
In this instance of a flexible manufacturing system, we consider four product type variations (A-D) using four different machines (1-4). The table below indicates for each pair of type and machine the production time interval and the desired production order. For example, the cell at row B and column Machine 1 indicates that processing product type B on Machine 1 takes between 9 and 11 minutes, and it is the second operation for product type B.

Product type	Machine 1	Machine 2	Machine 3	Machine 4
A	$[6, 10]; 1$	$[2, 5]; 2$	$[14, 20]; 3$	$[11, 14]; 4$
B	$[9, 11]; 2$	$[3, 6]; 1$	$[8, 10]; 4$	$[12, 17]; 3$
C	$[6, 9]; 4$	$[5, 8]; 3$	$[10, 15]; 1$	$[12, 16]; 2$
D	$[7, 11]; 2$	$[4, 5]; 4$	$[10, 15]; 1$	$[10, 14]; 3$

We wonder now how to optimally *schedule* the different product types on each machine so that the total time until all products have been processed (called make-span) is minimal. Here a schedule S gives for each product p and machine m the times $S(p, m)$ and $E(p, m)$, when product p starts and ends being processed by machine m. Of course the production formula (machine-orders and -times) must be met. Also, each machine can handle at most one product at the time. Different machines can process different products in parallel though.
Below you can find the screenshot of a Uppaal model of the problem. Here each product type is modelled as a timed automaton with a sequence of transitions reflecting the machine processing order. The processing times are modelled using constraints on a local clock. The fact that machines can process at most one product type is modelled using four Boolean variables used as semaphores. You can find the complete Uppaal model in Digital Exam in the file `ManufacturingSystem.xml`.



The exercise continuous on the next page.

Answer the following questions:

(a) Formulate as a UPPAAL query ϕ_1 that all 4 products can be completed (according to the ordering and timing constraints) within 60 minutes. Subsequently, check in UPPAAL whether ϕ_1 is satisfied, and if so, provide a schedule meeting the overall deadline of 60 minutes. You can illustrate the schedule using an Message Sequence Chart (MSC) chart for ϕ_1.

Hint: In the symbolic simulator of UPPAAL, you can save the generated MSC by right-clicking it and select "Export".

(b) Formulate as a UPPAAL query ϕ_2 that all 4 products can be completed (according to the ordering and timing constraints) within 40 minutes. Subsequently, check in UPPAAL whether ϕ_2 is satisfied, and if so, provide a schedule meeting the overall deadline of 40 minutes. You can illustrate the schedule using an MSC chart for ϕ_2.

(c) Use UPPAAL to find the minimum completion time. State what it is and give an optimal schedule achieving this minimum. You can illustrate the optimal schedule using an MSC chart obtained from UPPAAL.

(d) Use UPPAAL to find the maximum completion time, i.e., the time the system can guarantee all products are finished. State what it is and how you obtained this.

In the following modify the model in `ManufacturingSystem.xml` in three different (independent) ways.

(e) Due to a lack of maintenance, Machine 1 is running at half speed. That means that both the minimum and the maximum processing times for all product types are doubled. For example, the processing times for Product A on Machine 1 become $[12, 20]$. Modify the original model `ManufacturingSystem.xml` accordingly. Use UPPAAL to find the minimum and maximum completion time under this modification.

Please upload to Digital Exam your modified UPPAAL model in a file with name `ManufacturingSystem2.xml`.

(f) The manufacturing engineer wants to run Product A in the reverse order, i.e., starting with Machine 4 and ending with Machine 1. Modify the original model `ManufacturingSystem.xml` accordingly. Use UPPAAL to find the minimum and maximum completion time under this modification.

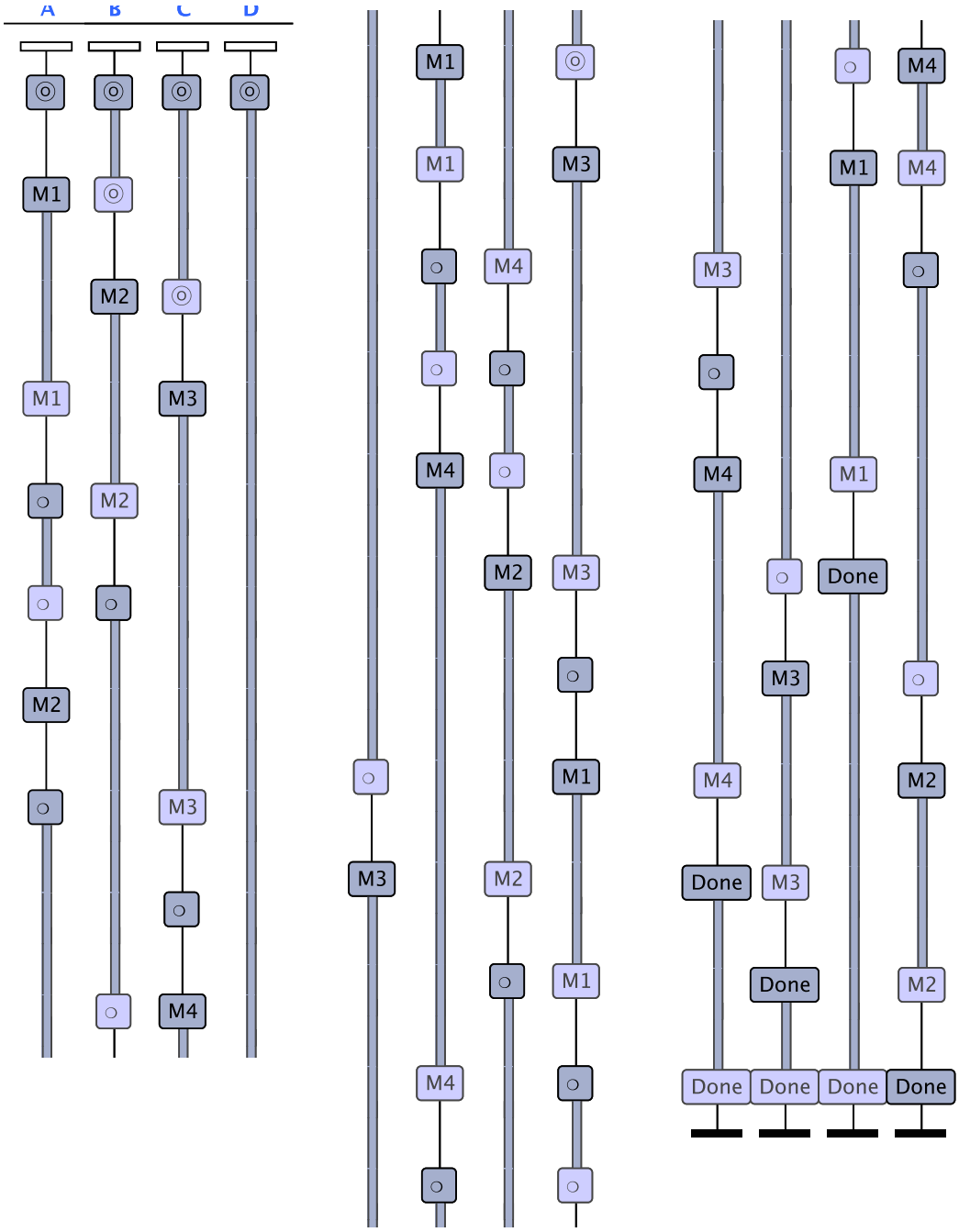
Please upload to Digital Exam your modified UPPAAL model in a file with name `ManufacturingSystem3.xml`.

(g) Machine 4 is the bottleneck, so a second one is installed. Even though each machine can only process one product at the same time, the two machines can work in parallel. Modify the original model `ManufacturingSystem.xml` accordingly. Use UPPAAL to find the minimum and maximum completion time under this modification.

Please upload to Digital Exam your modified UPPAAL model in a file with name `ManufacturingSystem4.xml`.

Solution sketch

(a) ϕ_1: `E<> A.Done and B.Done and C.Done and D.Done and time<=60`. The query is satisfied. The Message Sequence Chart of an optimal schedule (which also satisfies ϕ_1) is shown below.



(b) ϕ_2: `E<> A.Done and B.Done and C.Done and D.Done and time<=40`. The query is not satisfied.

(c) The minimum completion time is 55 minutes. One can obtain this by doing a binary search on the upper bound of the time using ϕ_1 as a basis, or one can ask UPPAAL to generate the fastest diagnostic trace for ϕ_1 and inspecting the Symbolic Simulator for the value of `time`. The MSC of an optimal schedule is shown in b).

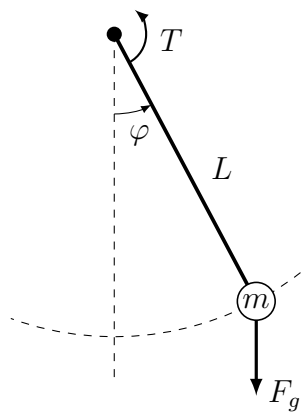
(d) The maximum completion time is 99 minutes. One can obtain this by doing a binary search on the upper bound of the time of the query `A<> A.Done and B.Done and C.Done and D.Done and time<=99`, or by doing a binary search on the lower bound of the time of the query `E[] A.Done and B.Done and C.Done and D. Done imply time>=99`, or by doing a binary search on the lower bound of the time of the query `A[] time>=100 imply A.Done and B.Done and C.Done and D. Done` and subtracting one from it. (There might be other ways as well.)

(e) The new minimum completion time is 65 minutes and the maximum completion time is 122 minutes.

(f) The new minimum completion time is 49 minutes and the maximum completion time is 95 minutes.

(g) We can model this by changing the Boolean variable `m4` into an integer variable with initial value of 2, i.e., the system starts with two available Machine 4's. Starting a product on Machine 4 decreases this counter, while finishing a product increases it. The new minimum completion time is 45 minutes and the maximum completion time is 93 minutes.

Exercise 6: Inverted pendulum 20 Points

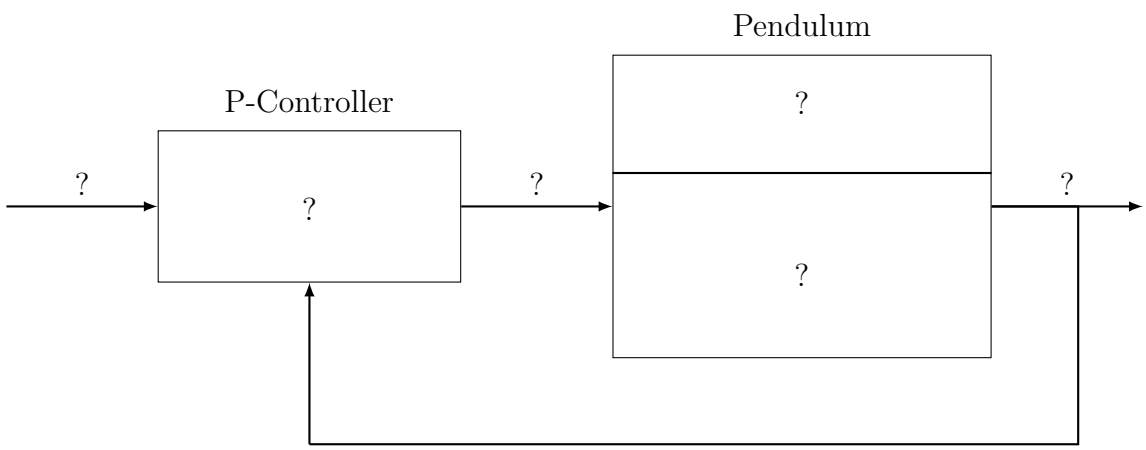


Consider the pendulum model given by

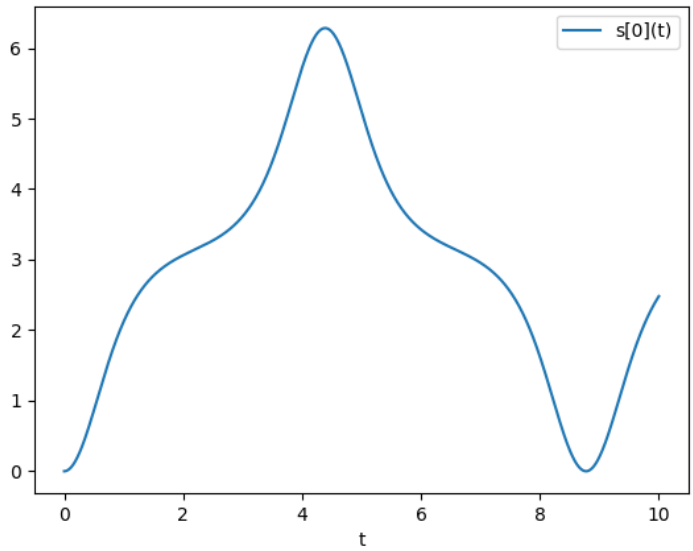
$$\frac{d}{dt}\varphi(t) = \nu(t) \qquad \frac{d}{dt}\nu(t) = -\frac{9.81}{1.5}\sin\varphi + \frac{1}{2\cdot 1.5}T(t),$$

where φ is the angle of the pendulum, ν the angular velocity and T the torque applied (for simplicity, we assume that the mass attached to the pendulum is 2 kg and the length of the pendulum is 1.5 m). Without any torque, the equilibrium point $\varphi = \pi$, i.e., the upright position, is unstable. We wish to make the upright position a stable point in order to obtain an inverted pendulum. To this end, we next devise a controller.

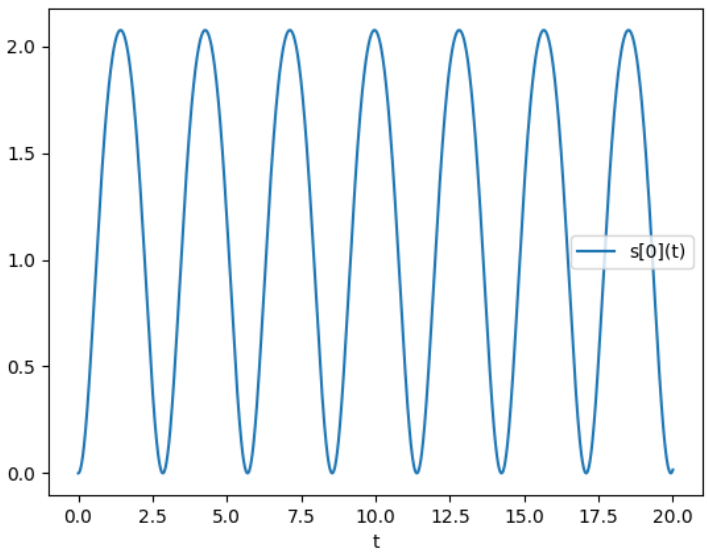
(a) Complete the diagram given below so that it describes a proportional controller of the pendulum in block diagram form.



(b) We first implemented the proportional controller with gain $K_p = 12$. Starting at rest from the bottom position $\varphi = 0$, we obtained the response of the system as shown in the figure below. Explain the system's response for this proportional controller. [Hint: Recall that $180° = \pi \approx 3.14$ and $360° = 2\pi \approx 6.28$]



(c) We changed the proportional controller to one with gain $K_p = 10$. Starting at rest from the bottom position $\varphi = 0$, we obtained the response of the system as shown in the figure below. Explain the system's response for this proportional controller. [Hint: Recall that $180° = \pi \approx 3.14$ and $360° = 2\pi \approx 6.28$]



For the following three questions, you will need to complete the provided Python script `NumericalSolverExam23.py`. You can find the Python script in Digital Exam. After finishing this exercise, please upload your modified Python script of the pendulum in a file with name `NumericalSolverExam23completed.py`, also in case you could only solve questions d) or e).

(d) Implement the `f` method in the provided Python script so that the overall program simulates the P-controlled pendulum from b). [Hint: You might want to use `math.sin()` and `math.pi` in Python for the sin function and the value of π, respectively.]

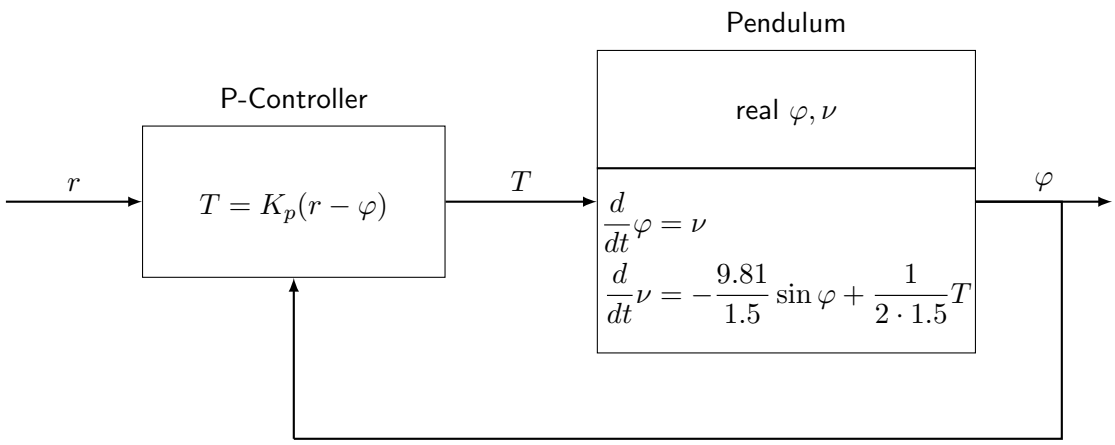
(e) Extend the P-controller from the foregoing exercise to a Proportional-Derivative (PD) controller. To this end, derive a formula for the derivative error and extend afterwards the method `f` in the Python script.

Tune and provide the proportional and derivative gains of the PD-controller so that the system is stabilized within 10 time units. Describe quality of your PD-controlled pendulum in terms of steady state error, settling time, overshoot and rise time.

(f) Extend the PD-controller from the foregoing exercise to a PID-controller. To this end, derive a formula for the integral error and extend afterwards the python method `f`. Tune and provide the proportional, derivative and integral gains of the PID-controller so that the system is stabilized within 10 time units. Describe the quality of your PID-controlled pendulum in terms of steady state error, settling time, overshoot and rise time.

.....Solution sketch .....

(a) The block diagram is completed as follows.

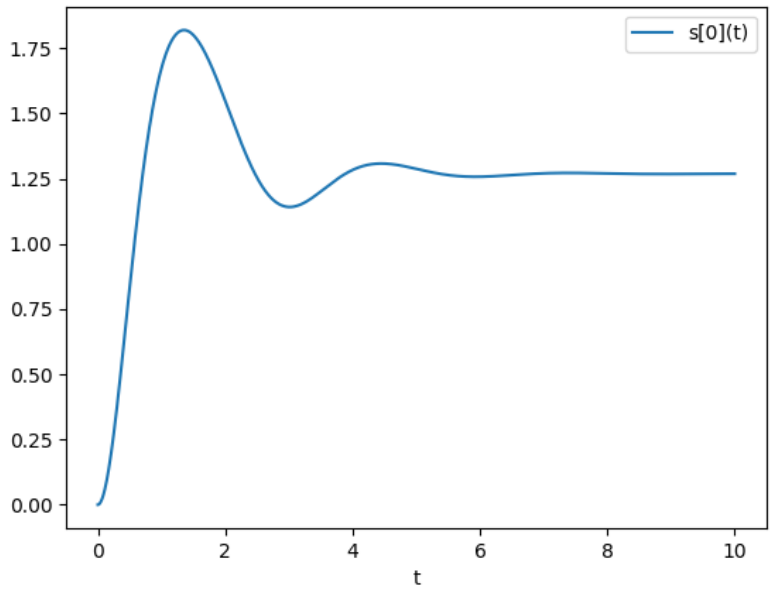


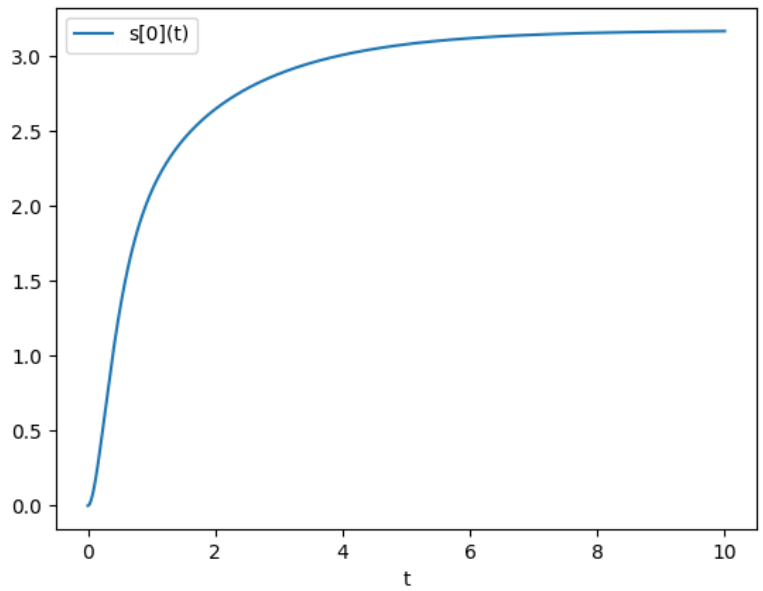
(b) The shown response is from a P-controller that provides just enough torque to bring the pendulum from the bottom position to the top position. But at the top position, the velocity is not precisely zero, hence the pendulum will overshoot. This overshoot is not immediately corrected by the P-controller in the top position, because the error is effectively zero. Therefore, without applying any torque on the pendulum, gravity will pull the pendulum down. When the pendulum goes down, the size of the error starts to increase again, and thus also the applied torque. Once the pendulum is down (but now with $\varphi = 2\pi$) the torque is strong enough to bring the pendulum back up. Now the process repeats indefinitely, as the model does not include friction.

(c) With a proportional gain $K_p = 10$, the P-controller is not able to lift the pendulum to the top-right position. It will start to oscillate between $0 \leq \varphi \leq \varphi_{max} < \pi$, where the maximum angle φ_{max} depends on the size of the proportional gain.

(d) For code, please see (f).

(e) For code, please see (f). Note that the derivative error is equivalent to $\frac{d}{dt}e(t) = \frac{d}{dt}(r - \varphi(t)) = -\nu(t)$. The figure below is obtained with $K_p = 10$ and $K_d = 5$. By considering an extension to a PD-controller, we can eliminate the oscillation of a pure P-controller and gain an asymptotically stable controlled system with, in our case, a settling time of 6 time units, which is below the limit of 10 time units. Unfortunately, we are left with a steady-state error (the higher K_p, the smaller this error). This can be addressed by adding an integral part to the control, yielding a PID-controller. Depending on the values of K_p and K_d, one might obtain a response with some overshoot and a quick rising time (see the figure for $K_p = 10$ and $K_d = 5$), or one without overshoot and rising time.



(f) Instead of being given by $I(t) = \int e(t)\mathrm{d}t$, the integral error enjoys the differential equation $\frac{d}{dt}I(t) = e(t) = r - \varphi(t)$. This results in an additional state variable (so we need to adapt the initial state declaration in `main` as well!). Using a PID-controller with $K_p = 30, K_d = 20$ and $K_i = 0.1$, whose code is given below, we can achieve satisfying performance by eliminating overshooting and steady-state errors, and having a rise time = settling time of roughly 8 time units, all within the required time frame. (Note that the PD-controller is obtained by letting $K_i = 0$, while the P-controller if $K_i = K_d = 0$.)

```python
def f(x, t):
    # Implements the simplified pendulum model
    m = 2
    g = 9.81
    l = 1.5
    r = math.pi

    Kp = 30
    Kd = 20
    Ki = 0.1
    T = Kp * (r - x[0]) - Kd*x[1] + Ki * x[2]

    der_phi = x[1]
    der_v = -(g/l)*math.sin(x[0]) + T/(m*l)
    der_I = r - x[0]
    return [der_phi, der_v, der_I]

def main():
    x0 = [0, 0]  # Initial state for answer e) and f), as provided in given script.
    x0 = [0, 0, 0]  # Initial state for answer g).
    ...
```